

MG后台SDK配置梳理

新版本SDK配置参数

按 6 个配置模块分别保存 JSON：payment、login、agreement、thirdPartyData、advertising、switches。合规、客服工具继续使用旧接口，不进入这套新结构。

一、统一存储方式

使用一张模块配置表，而不是每个模块单独建表：

代码块

```
1 CREATE TABLE sdk_module_config (  
2     id BIGINT NOT NULL COMMENT '雪花ID',  
3     app_id BIGINT NOT NULL COMMENT '游戏/应用ID',  
4     module_code VARCHAR(32) NOT NULL COMMENT '配置模块编码',  
5     schema_version INT NOT NULL DEFAULT 1 COMMENT '模块JSON结构版本',  
6     config_json LONGTEXT NOT NULL COMMENT '模块配置JSON',  
7     configured TINYINT(1) NOT NULL DEFAULT 0 COMMENT '是否已满足模块完整配置要求',  
8     version BIGINT NOT NULL DEFAULT 1 COMMENT '配置修改版本号',  
9     created_by BIGINT DEFAULT NULL COMMENT '创建人用户ID',  
10    updated_by BIGINT DEFAULT NULL COMMENT '最后修改人用户ID',  
11    create_time DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),  
12    update_time DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)  
13              ON UPDATE CURRENT_TIMESTAMP(3),  
14    PRIMARY KEY (id),  
15    UNIQUE KEY uk_app_module (app_id, module_code)  
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
17 COMMENT='应用级SDK模块配置';
```

接口可以统一为：

代码块

```
1 PUT /apps/{appId}/sdk-config/{moduleCode}
```

```
1  {
2    "schemaVersion": 1,
3    "status": "DRAFT",
4    "config": {}
5  }
```

整体上，一张模块表加 JSON 字段适合这批配置：字段差异大、平台会持续增加，而且多数数据只会整块读写。

合规和客服工具继续保留原来的独立表和接口即可。

二、支付 payment

JSON:

代码块

```
1  {
2    "notify_url": "https://api.example.com/payment/delivery",
3    "channels": {
4      "apple": {
5        "sharedSecret": "apple-shared-secret"
6      },
7      "google": {
8        "googleServerConfig": {
9          "fileName": "google-payment-config.json",
10         "secret": {
11           "type": "service_account",
12           "project_id": "top-of-mg-f2cbc",
13           "private_key_id": "d462e1e69f42e28e5f281272f0b14c48f6a65ce8",
14           "private_key": "-----BEGIN PRIVATE KEY-----
\nMIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQD0cfe5Iq2ukfik\nXT31Q9kUmVk
MB/jY9YcC1laionpRZJr162LXjb82k4FAu7ZA2dL5iSa/kp/+X41T\n0/MGKyyTKjH1Ymkv8s25Rckb
jnvN EK8+ZL1wIENngvSgr00X73fymSyexMpmDIXw\nnxODBP+zKMKCGt5yRLvaYHeZy6DA4DoLKHNr5v
du0kjdBesQtTt0xPQV2ql049o1s\nHwFJa0UTuc81s/wVOFeDe1QdHVOQyrHyIEEHmsgeeLIZBrCY7z
U0unXji6hTM4B/\nh6HXhTa07MEEoiSWvC2GVFUODYhiurVkvux2N+inhp5dgVGJ9os+rzLY6WqD2J
s\nXEhS1rUFAGMBAAECggEAEZRVtuBNwWoVz6zsSntjBXRi1ZUwz/zu9h1ie5BRNczQh\nIh/G7oej5F
u2UpV3CGr/wBp1RCFNwNZVkdL6C3Urhxfi0jOW2F2d0zkK1+JcJr9\ns740+cV6a6E2GcWngaoKH9r
HLUhrS3BoVvt3l3gvDI54023tNjN6IAKn94SUM6WZ\nniFTN4d/aIiN6//o3p2cuHiEmXTFCJ1F5v2/q
BobpNM4SEPMSKROAub4ht+aWc4cJ\nnmV2DWLqfWM2/oQ0T02leNBec3on9NtW0n8e6XYmz1ps2jy6RU
4VGCXjfi15pGlvF\nnTyqo0vMge3BIkfd0AZdH2yvVW1cjibh0y03xLza0IwKBgQD3kuTbr0Cumd4Gfm
o7\n/6TJY+Dxah7Ipr/kR9Wsz5J2f4ZJKd8Ys/NSbP9BKzs8WoUayFLNdUsEKIkuy65N\nnQKL1moY4E
VDjWRZu28rJSiguq7NYUz/jts2J9w5QTgCViHZZu6Q/76xjEzi1cHIF\nn/lrfzCa0lMr1Xsi4ZVeCZx
4YVwKBgQDVeLiF5quXhRAUs2VzrEA6FghDyOW/2HKZ\nnRCDKqnaJv7y+rw0b0J/3YltaLIqyP9xxjJ0
AUuzUXxKh10z/NYWEzeUKEJOKBPGj\nngfs00exQzVLI32wJuWpEx16MwoYaaa0YJnl0bPUswlt+7Kss
```

```

4satksyvbS4IL7SK\nr1uhWLN0AwKBgAc9373hUXYKACmWdwsMPki5yDGnlgDuh7xosYJdi/41Wrs3I
1R4\npwxpLco9Nn1cdHpFE874uFoFyYV55b5SvrGKdJSdRQ1xZKuXeEYNCKHbOeLChKLn\njPmVJXEJ
5xB4rmWPjI1mOFepj0YXZ05x8zyKnzIafhlRtX0/WyDwXTv/AoGBAKzT\nzvjBl1yDAOXRLCqBY4Jjg
g93H+MhV00hvb3KdRLEIGE6jQFR02hWPrN1jiYEV5x\nnDE9N0+XgjxH6j+DzMQUj0PLcFscgV44cVw
gdi0imQwbUNADvd4T1sKvfdYRUepld\nV0mYqSqytCbfb3qoo1e1yTeh+imwjcFWTyYkHfqHAoGAYCz
CJ0hy9scw9RdUb5eB\nnudYKUsk12/e6VmZoVnxll6vXuqt5JJ54mFEA9drfOS7UhU9NMvZ4820Pgb79
Ak49\nT8FPJfYpVr1xDhUFXB/bqq7EN3myIutpLEDVV8wWwMXpg7zkhGg6PSdFbn3xxrYq\nT3z02EU
bmigv0xtqtI/K2GQ=\n-----END PRIVATE KEY-----\n",
15         "client_email": "top-of-mg-pay-test@top-of-mg-
f2cbc.iam.gserviceaccount.com",
16         "client_id": "115132823542864895441",
17         "auth_uri": "https://accounts.google.com/o/oauth2/auth",
18         "token_uri": "https://oauth2.googleapis.com/token",
19         "auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
20         "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/top-of-mg-pay-test%40top-of-
mg-f2cbc.iam.gserviceaccount.com",
21         "universe_domain": "googleapis.com"
22     }
23 }
24 },
25     "one_store": {
26         "license_key": "one-store-license-key",
27         "clientId": "one-store-client-id"
28     }
29 }
30 }

```

字段规则：

- `channels` 至少包含一个完整渠道配置。
- JSON 中出现某个渠道时，该渠道的所有字段必须完整。
- 文件不要以 Base64 放入配置 JSON，只保存上传文件的引用。

三、登录 login

页面共有 12 种登录方式：

使用带 `type` 的数组，避免页面内部 `selected + values` 两份状态不一致：

代码块

```

1  {
2      "methods": [

```

```
3    {
4      "type": "guest"
5    },
6    {
7      "type": "email"
8    },
9    {
10     "type": "google",
11     "params": {
12       "common": {
13         "webClientId": "xxx.apps.googleusercontent.com",
14         "clientSecret": "secret"
15       },
16       "ios": {
17         "iosClientId": "xxx.apps.googleusercontent.com",
18         "iosUrlScheme": "com.googleusercontent.apps.xxx",
19         "iosBundleId": "com.company.game"
20       },
21       "android": {
22         "androidClientId": "xxx.apps.googleusercontent.com",
23         "androidPackageName": "com.company.game",
24         "androidSha1": "12:34:..."
25       }
26     }
27   }
28 ]
29 }
```

各登录方式参数：

登录type	共有参数	iOS独有	Android独有
guest	无	无	无
email	无	无	无
apple	teamId、keyId、privateKey	iosBundleId	androidServicesId、androidRedirectUri
google	webClientId、clientSecret	iosClientId、iosUrlScheme、iosBundleId	androidClientId、androidPackageName、androidSha1
kakao	nativeAppKey	iosBundleId、iosUrlScheme	androidPackageName、androidKeyHash
facebook	appId、clientToken	iosBundleId、iosUrlScheme	androidPackageName、androidKeyHash
twitter	apiKey、apiSecret、callbackUrl	无	无
snapchat	clientId、redirectUri	iosBundleId	androidPackageName
line	channelId、channelSecret	iosBundleId、iosUniversalLink	androidPackageName
naver	clientId、clientSecret	iosUrlScheme	androidPackageName
tiktok	clientKey、clientSecret	iosUrlScheme	androidRedirectScheme、androidPackageName
discord	clientId、clientSecret、redirectUri	无	无

四、协议与隐私 agreement

页面操作是按协议分组分别进行配置的，服务端保存数据也要按协议分组进行存储，每个协议分组下为多个语种配置

代码块

```
1  {
2    "groups": [
3      {
4        "name": "协议分组名称",
5        "defaultLanguage": "en",
6        "locales": {
7          "en": {
8            "name": "Privacy Policy",
9            "url": "https://example.com/privacy/en"
10         },
11         "zh-CN": {
12           "name": "隐私政策",
13           "url": "https://example.com/privacy/zh-cn"
```

```
14     }
15     }
16   }
17 ]
18 }
```

语言映射：

页面名称	JSON key
英语	en
简体中文	zh-CN
繁体中文	zh-TW
韩语	ko
阿拉伯语	ar

五、三方数据 `thirdPartyData`

JSON：

代码块

```
1  {
2    "platforms": [
3      {
4        "type": "firebase",
5        "config": {
6          "androidFile": {
7            "firebase_url": "images/a543eba17c3c467aa7c14af5bc9c2765.json",
8            "firebase_name": "google-services.json"
9          },
10         "iosFile": {
11           "firebase_url": "images/72eea860c6f140b6a1c18c07933f3fd1.plist",
12           "firebase_name": "GoogleService-Info.plist"
13         }
14       }
15     ],
16     {
17       "type": "appsflyer",
18       "config": {
19         "dev_key": "appsflyer-dev-key",
20         "apple_app_id": "6748123091"
21       }
22     }
23   ]
24 }
```

```

22     },
23     {
24         "type": "adjust",
25         "config": {
26             "appIdentifier": "adjust-app-token",
27             "login_id": "event-id",
28             "sign_up_id": "event-id",
29             "tutorial_begin_id": "event-id",
30             "tutorial_complete_id": "event-id",
31             "unlock_achievement_id": "event-id",
32             "level_up_id": "event-id",
33             "achievementEvent": "event-id",
34             "purchase_id": "event-id",
35             "share_id": "event-id",
36             "earnCurrencyEvent": "event-id",
37             "spendCurrencyEvent": "event-id",
38             "level_start_id": "event-id",
39             "level_end_id": "event-id"
40         }
41     }
42 ]
43 }

```

六、广告变现 advertising

代码块

```

1  {
2      "advertising": {
3          "admob": {
4              "admob_app_id": ""
5          },
6          "applovin": {
7              "max_sdk_key": ""
8          },
9      }
10 }

```

七、运营配置策略

运营配置引入了发行策略字段，所以需要根据运营策略分别进行数据存储

当前已支持的根据国家策略存储的数据：

1.登录方式排序和开关

需要优化才支持国家策略存储的数据：

- 1.GDPR合规隐私协议开关，目前是开关为渠道全局开关，开关开启，如果是GDPR地区就会弹GDPR合规弹窗，不是GDOR就弹正常协议弹窗。（这个GDPR地区配置是SDK的全局配置，需改到渠道配置里）
- 2.年龄获取与家长控制开关，目前是开关为渠道全局开关，开关开启，如果是需要获取年龄和家长控制的地区就会走此功能（这个需要家长控制的地区配置是SDK的全局配置，需改到渠道配置里）
- 3.开关配置，开关配置当前是渠道全局开关，开关开启关闭影响整个渠道。（这个需要额外增加）

新版本SDK配置接口梳理

一、配置中心参数配置查询、保存接口

1. 配置中心7个模块的参数展示需要查询接口支持，此查询接口返回所有的参数配置
2. 单个模块进行参数配置保存时，需要修改接口支持，此修改接口保存所有更改，如果已创建渠道，同步修改渠道配置的参数

二、生成及运营中心渠道创建、查询、修改接口

1. 渠道查询接口：查询所有渠道配置，此接口返回渠道所有配置信息，渠道类型、SDK能力、包名、更新时间
2. 渠道创建接口：创建渠道时需要将配置中心的参数写到渠道配置相关的数据库表里 [🔗 现有MG后台参数配置接口梳理](#)
3. 渠道修改接口：支持修改渠道配置，更新渠道配置信息，包名、SDK能力

三、运营配置查询、修改接口

运营配置引入了发行策略字段，所以需要根据运营策略分别进行数据存储

同样的运营配置页面的展示目前仅支持针对登录方式进行修改

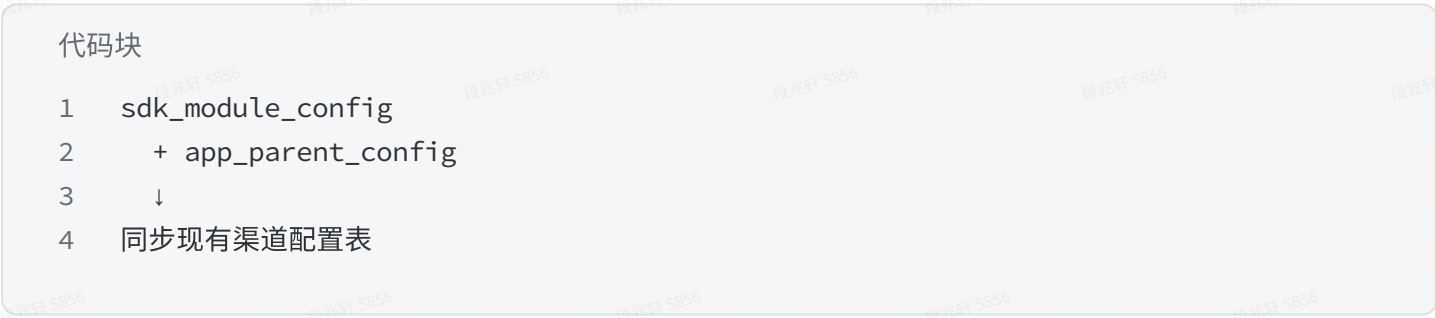
MG 后台 SDK 配置接口方案

整体数据模型

采用“应用级配置源 + 现有渠道配置表”的结构。



渠道创建或应用配置更新时：



`sdk_module_config` 是应用级唯一 JSON 配置源，不再为每个渠道复制相同 JSON。

接口方法约定

- GET：只用于查询，不修改数据库。
- POST：用于保存、创建和修改数据。

配置中心范围

配置中心包含：

1. 支付 `payment`
2. 登录 `login`
3. 协议与隐私 `agreement`
4. 合规 `compliance`

5. 三方数据 `thirdPartyData`

6. 广告变现 `advertising`

7. 客服工具 `customerService`

存储规则：

- `payment/login/agreement/thirdPartyData/advertising` 保存到 `sdk_module_config`。
- `compliance` 只包含 KWS 参数和年龄阈值，保存到 `app_parent_config`，新增年龄阈值。
- GDPR、年龄家长控制开关不进入应用级配置，渠道创建后使用渠道表默认值 `false`。
- `customerService` 查询和保存继续使用原 help-service 接口。

配置中心查询接口

代码块

```
1 GET /customer/topsdk/console/meetgames/sdk-config/getAppConfig?appId={appId}
```

返回：

代码块

```
1  {
2    "appId": "791251136341225472",
3    "modules": {
4      "payment": {
5        "configured": true,
6        "version": 3,
7        "config": {}
8      },
9      "login": {
10       "configured": false,
11       "version": 2,
12       "config": {},
13       "missingFields": [
14         "methods[google].params.common.webClientId"
15       ]
16     },
```

```

17     "compliance": {
18         "configured": true,
19         "config": {
20             "ageThreshold": 13,
21             "kws": {
22                 "clientId": "kws-client",
23                 "clientSecret": "*****",
24                 "verifySecret": "*****",
25                 "redirectUrl": "https://api.example.com/kws/callback"
26             }
27         }
28     },
29     "customerService": {
30         "configured": true,
31         "config": {}
32     }
33 }
34 }

```

单模块局部保存接口

代码块

```

1  POST /customer/topsdk/console/meetgames/sdk-config/saveModuleConfig
2  Content-Type: application/json

```

请求必须明确传入 `moduleCode` :

代码块

```

1  {
2      "appId": "791251136341225472",
3      "moduleCode": "payment",
4      "config": {
5          "notifyUrl": "https://api.example.com/payment/notify"
6      }
7  }

```

`moduleCode` 支持:

- 1 payment
- 2 login
- 3 agreement
- 4 compliance
- 5 thirdPartyData
- 6 advertising

服务端通过 `moduleCode` :

- 确认写入哪个模块。
- 选择对应的字段合并规则。
- 选择对应的 key、value 类型和格式校验器。
- 选择渠道旧表适配器。
- 确认数组使用哪个业务字段作为唯一标识。

返回合并后的完整配置：

代码块

```
1  {
2    "moduleCode": "payment",
3    "configured": true,
4    "version": 4,
5    "config": {
6      "notifyUrl": "https://api.example.com/payment/notify",
7      "channels": {
8        "apple": {
9          "sharedSecret": "*****"
10       }
11     }
12   },
13   "missingFields": [],
14   "syncedChannelCount": 3,
15   "updatedAt": "2026-08-11 10:10:00"
16 }
```

局部更新规则

1. 未传字段不修改

数据库原配置：

代码块

```
1  {
2    "notifyUrl": "https://old.example.com",
3    "channels": {
4      "apple": {
5        "sharedSecret": "old-secret"
6      },
7      "google": {
8        "googleServerConfig": {
9          "fileId": "100"
10       }
11     }
12   }
13 }
```

请求：

代码块

```
1  {
2    "moduleCode": "payment",
3    "config": {
4      "notifyUrl": "https://new.example.com"
5    }
6  }
```

保存结果只修改 `notifyUrl`，支付渠道配置保持不变。

2. 对象递归合并

请求：

代码块

```
1  {
2    "moduleCode": "payment",
3    "config": {
4      "channels": {
5        "apple": {
6          "sharedSecret": "new-secret"
7        }
8      }
9    }
```

```
10 }
```

只修改 Apple 密钥，不覆盖 Google 配置。

3. `null` 表示删除

代码块

```
1 {
2   "moduleCode": "payment",
3   "config": {
4     "notifyUrl": null
5   }
6 }
```

表示删除 `notifyUrl`。

必填且不允许删除的字段传 `null` 时返回参数错误。

年龄阈值不能传 `null`，需要重置时传 `0`。

4. 空字符串是有效输入

空字符串表示明确保存为空字符串，不等同于未传。

是否允许为空，由对应 `moduleCode` 的字段校验器决定。

5. 数组按照业务 ID 合并

数组	合并字段
login.methods	type
agreement.groups	groupId
thirdPartyData.platforms	type

只修改 Google 登录：

代码块

```
1 {
2   "appId": "791251136341225472",
3   "moduleCode": "login",
4   "config": {
5     "methods": [
6       {
7         "type": "google",
```

```

8      "params": {
9        "common": {
10         "webClientId": "new-client-id"
11       }
12     }
13   }
14 ]
15 }
16 }

```

其他登录方式不变。

新增协议组没有 `groupId` 时，由后端生成并在响应中返回。

删除数组项：

代码块

```

1  {
2    "moduleCode": "login",
3    "config": {
4      "methods": [
5        {
6          "type": "google",
7          "_delete": true
8        }
9      ]
10   }
11 }

```

`_delete` 只参与请求合并，不存入数据库。

保存与校验流程

代码块

- 1 接收 appId、moduleCode、config
- 2 → 校验应用所有权
- 3 → 根据 moduleCode 加载模块校验器
- 4 → 校验本次提交的 key 和 value
- 5 → 读取数据库原配置
- 6 → 递归合并本次 config
- 7 → 检查合并后的完整性
- 8 → 保存应用配置
- 9 → 同步已有渠道

校验结果：

- 未知 key：拒绝保存。
- value 类型、格式、枚举非法：拒绝保存。
- 文件引用不属于当前 appId：拒绝保存。
- 只是缺少必填字段：允许保存，返回 `configured=false`。
- 配置完整：返回 `configured=true`。

在选择对应 SDK 能力时，要求模块 `configured=true`。

合规模块

配置中心合规只允许：

代码块

```
1  {
2    "ageThreshold": 13,
3    "kws": {
4      "clientId": "kws-client",
5      "clientSecret": "secret",
6      "verifySecret": "verify-secret",
7      "redirectUrl": "https://api.example.com/kws/callback"
8    }
9  }
```

明确不包含：

代码块

```
1  gdprEnabled
2  ageParentalEnabled
3  gdprCheck
4  ageParentalCheck
```

如果合规请求出现这些字段，服务端返回“不支持的配置字段”。

合规局部保存示例

只修改年龄阈值：

代码块

```
1  {
2    "appId": "791251136341225472",
3    "moduleCode": "compliance",
4    "config": {
5      "ageThreshold": 15
6    }
7  }
```

只修改 KWS Redirect URL：

代码块

```
1  {
2    "appId": "791251136341225472",
3    "moduleCode": "compliance",
4    "config": {
5      "kws": {
6        "redirectUrl": "https://api.example.com/new-kws/callback"
7      }
8    }
9  }
```

合规保存不写入 `sdk_module_config`，而是根据 `appId + parentType=0` 更新 `app_parent_config`。

KWS 密钥规则：

- 未传：不修改。
- 传 `"*****"`：保持原密钥。
- 传新值：更新密钥。
- 传 `null`：清空密钥。

年龄阈值存储与同步

app_parent_config 新增：

代码块

```
1 ALTER TABLE app_parent_config
2 ADD COLUMN age_threshold TINYINT UNSIGNED NOT NULL DEFAULT 0
3 COMMENT '配置中心年龄阈值，创建或更新渠道时同步到channel';
```

实体新增：

代码块

```
1 private Integer ageThreshold;
```

规则：

- 年龄阈值允许 0~18。
- app_parent_config.age_threshold 只用于应用级配置存储。
- SDK 实际配置仍读取 channel.age_threshold。
- 保存年龄阈值时，如果应用已有渠道，同步更新全部渠道。
- 应用没有渠道时，只保存 app_parent_config，不自动创建渠道。
- 创建渠道时，从 app_parent_config 读取并写入新渠道。
- 未配置时使用 0。

渠道创建后的默认值：

代码块

```
1 channel.gdpr_check = false
2 channel.age_parental_check = false
3 channel.age_threshold = app_parent_config.age_threshold
```

不从配置中心同步 GDPR 和年龄家长控制开关。因为配置中心只进行配置参数的操作，没有开关操作设计

渠道同步方式

不保存渠道模块 JSON 副本。

配置中心保存

代码块

```
1  更新 sdk_module_config
2    → 查询 appId 下全部 channel
3    → 根据 moduleCode 调用渠道同步适配器
4    → 更新现有渠道配置表
```

主要投影目标：

模块	渠道表
payment	channel、channel_payment
login	channel_auth
agreement	channel_agreement
thirdPartyData	channel_third_data_platform
advertising	channel_ad_config
compliance.ageThreshold	channel.age_threshold

新配置字段如果没有对应旧表字段：

- 保留在 sdk_module_config.config_json。
- 新版 SDK 生成逻辑根据 channel.app_id 直接读取应用模块配置。
- 不为了复制相同 JSON 增加渠道模块表。
- 如果将来确实需要渠道级差异，再单独设计渠道覆盖配置。

新建渠道

代码块

```
1  创建 channel
2    → 读取 sdk_module_config
3    → 初始化现有渠道配置表
4    → 读取 app_parent_config.age_threshold
```

- 5 → 写入 `channel.age_threshold`
- 6 → GDPR和年龄家长控制开关使用默认false

文件上传接口

代码块

- 1 POST `/customer/topsdk/console/meetgames/sdk-config/uploadConfigFile`
- 2 `Content-Type: multipart/form-data`

参数:

代码块

- 1 `appId`
- 2 `moduleCode`
- 3 `usageCode`
- 4 `file`

返回:

代码块

```
1 {  
2   "fileId": "9000001",  
3   "fileName": "google-services.json",  
4   "contentType": "application/json",  
5   "size": 2048,  
6   "sha256": "..."  
7 }
```

模块配置只保存:

代码块

```
1 {  
2   "fileId": "9000001",  
3   "fileName": "google-services.json"  
4 }
```

渠道及运营配置接口

查询渠道

代码块

```
1 GET /customer/topsdk/console/meetgames/sdk-config/getChannels?appId={appId}&pageNo=1&pageSize=20
```

创建渠道

代码块

```
1 POST /customer/topsdk/console/meetgames/sdk-config/createChannel
```

修改渠道

代码块

```
1 POST /customer/topsdk/console/meetgames/sdk-config/updateChannel
```

查询运营配置

代码块

```
1 GET /customer/topsdk/console/meetgames/sdk-config/getOperationConfig?appId={appId}&channelId={channelId}
```

保存运营配置

代码块

```
1 POST /customer/topsdk/console/meetgames/sdk-config/saveOperationConfig
```

渠道能力、渠道运行开关保存在 `sdk_channel_package_config`；多发行策略保存在 `sdk_channel_operation_strategy`。

数据库表

sdk_module_config

代码块

- 1 字段含义
- 2 id 主键，使用项目现有雪花 ID
- 3 app_id 游戏 ID，对应 app.id
- 4 module_code 模块编码，用于选择 JSON 合并和校验规则
- 5 schema_version JSON 参数结构版本，后续新增、重命名参数时用于兼容
- 6 config_json 当前模块合并后的完整 JSON
- 7 configured 是否已经满足完整配置条件；允许保存不完整配置时为 0
- 8 version 每次有效修改后加 1，可用于操作记录和并发控制
- 9 created_by 首次创建配置的用户
- 10 updated_by 最后修改配置的用户
- 11 create_time 创建时间
- 12 update_time 最后修改时间
- 13
- 14 UNIQUE(app_id, module_code)

只保存：

代码块

- 1 payment
- 2 login
- 3 agreement
- 4 thirdPartyData
- 5 advertising

app_parent_config

继续保存 KWS，新增：

代码块

- 1 age_threshold

sdk_config_file

保存配置文件引用、OSS objectKey、文件类型、大小、SHA-256 和 appId 所有权。

sdk_channel_package_config

保存渠道 SDK 能力、渠道包级运行开关和更新时间。

sdk_channel_operation_strategy

保存渠道下的多条发行策略、国家范围、登录排序、协议组和能力开关。

明确不创建：

代码块

```
1 sdk_channel_module_config
```

验收测试

- 请求未传 `moduleCode` 时保存失败。
- 不支持的 `moduleCode` 保存失败。
- 只传一个普通字段时，其他字段保持不变。
- 嵌套对象只修改请求中出现的叶子字段。
- `null` 能删除允许删除的字段。
- 空字符串按显式值保存并执行字段校验。
- 登录方式按照 `type` 合并，不覆盖其他登录方式。
- 协议组按照 `groupId` 合并。
- 三方数据平台按照 `type` 合并。
- `_delete=true` 能删除指定数组项。
- 缺少必填参数时允许保存，但 `configured=false`。
- 非法 URL、类型错误和未知 key 不允许保存。
- 合规模块出现 GDPR 或家长控制开关时保存失败。
- 只修改年龄阈值时不覆盖 KWS 参数。
- 只修改一个 KWS 字段时不覆盖其他 KWS 字段。
- 没有渠道时保存年龄阈值不会创建渠道。
- 有渠道时年龄阈值同步到全部渠道。
- 创建渠道时从 `app_parent_config` 复制年龄阈值，GDPR 和家长控制开关保持默认关闭。
- 客服工具继续使用原查询和保存接口。